

Backend

The Job Tracker API is a RESTful backend service that allows authenticated users to track job applications, manage application statuses, and receive automated email reminders for follow-ups.

This backend is designed to support a full-stack web application and demonstrates:

- Secure JWT-based authentication
- Role-based data ownership
- CRUD operations
- Background jobs (cron)
- Email notifications

Tech Stack

- Backend
- Node.js
- Express.js
- MongoDB
- Mongoose

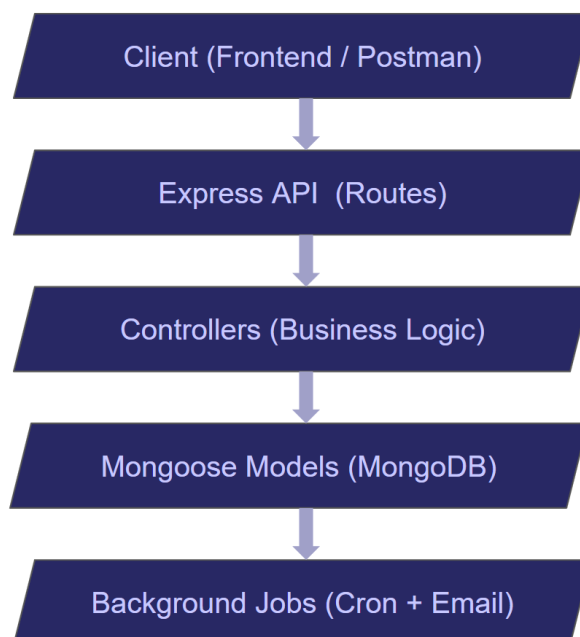
Auth

- JSON Web Tokens (JWT)
- Bcrypt

Utilities

- node-cron (scheduled jobs)
- nodemailer (email delivery)

Architecture



Authentication Flow

1. User registers or logs in
2. Server issues a JWT
3. Client stores the token
4. Token is sent in the Authorization header
5. Protected routes validate token
6. User identity is attached to req.user

Auth Endpoints

Register User (POST /api/auth/register)

Request

```
{
  "name": "Name",
  "email": "name@example.com",
  "password": "password123"
}
```

Response

```
{
  "_id": "64fa...",
  "name": "Name",
  "email": "name@example.com",
  "token": "JWT_TOKEN_HERE"
}
```

Login User (POST /api/auth/login)

Request

```
{
  "email": "luise@example.com",
  "password": "password123"
}
```

Response

```
{
  "_id": "64fa...",
  "name": "Luis",
  "email": "luise@example.com",
  "token": "JWT_TOKEN_HERE"
}
```

Application Schema

```
{
  user: ObjectId (User reference),
  company: String,
  role: String,
  status: String,
  followUpDate: Date,
  reminderSent: Boolean,
  notes: String,
  createdAt: Date,
  updatedAt: Date
}
```

Application Endpoints

Create Application (POST /api/applications)

Request

```
{
  "company": "Amazon",
  "role": "Software Engineer",
  "status": "Applied",
  "followUpDate": "2026-01-08",
  "notes": "Reached out to recruiter"
}
```

Response

```
{
  "_id": "65ab...",
  "company": "Amazon",
  "role": "Software Engineer",
  "status": "Applied",
  "followUpDate": "2026-01-08T00:00:00.000Z",
  "reminderSent": false
}
```

Get User Applications (GET /api/applications)

- Returns only applications owned by the logged-in user

Response

```
[
  {
    "_id": "65ab...",
    "company": "Amazon",
```

```
    "role": "Software Engineer",
    "status": "Interview",
    "followUpDate": "2026-01-08T00:00:00.000Z"
  }
]
```

Update Application (PUT /api/applications/:id)

- Only the owner can update the application

Request

```
{
  "status": "Offer"
}
```

Authorization Rules

- Users can only access their own applications
- Cross-user access is blocked
- Invalid ObjectIds return 404

Email Reminder System

Purpose: Automatically notify users to follow up on applications when the follow-up date is reached.

- server/jobs/reminder.job.js
- Runs daily via cron
- Finds applications where:
 - followUpDate <= today
 - reminderSent === false

Email Logic

Each reminder:

- Sends an email to the application owner
- Includes company and role
- Marks reminderSent = true to prevent duplicates

Security Considerations

- Passwords are hashed using bcrypt
- JWT secret stored securely
- Protected routes enforce ownership
- Emails use app passwords (not raw credentials)